UNIVERSITY OF IBADAN

DEPARTMENT OF COMPUTER SCIENCE

IFT302: WEB DESIGN AND DEVELOPMENT

VEHICLE SERVICE BOOKING AND PAYMENT PLATFORM

(ui-vsb)

Group Project Report

Prepared for the University of Ibadan Maintenance Directorate

Date: 23-08-2026

IFT302

Team Members

S/N	Name	Part Worked On	Matric Number
1	Nwachukwu Victor Efemena	Backend Development	250380
2	Muhammed Awwal Mumeenat	Frontend Development	256647
3	Akano Hazeem Olamilekan	Authentication	250351
4	Olanipekun Happiness	Backend Development	250396
5	Oyeniran Jibola Gbolahan	Project Report	250405
6	Ayedun Caleb Ephraim	Project Report	250501
7	Sunday Emmanuel	Testing	250559
8	Adejoro Victory Adesewa	Project Report	256566
9.	Adesesan Toluwani Oluwadarasimi	Payment Simulation	250342

GitHub URL: https://github.com/DvectorPrime/ui-vsb

IFT302

Table of Contents

1. Introduction **1**
2. Problem Statement **2**
3. Objectives **3**
4. System Requirements **4**
4.1 Functional Requirements 4
4.2 Non-Functional Requirements 4
4.3 Software Requirements 5
4.4 Hardware Requirements 5
5. Use-Case Diagram **6**
6. System Architecture **7**
7. Database / ER Diagram **9**
8. Database Tables **10**
9. Important Interface Screenshots **13**
9.1 Customer Interface 13
9.2 Administrator Interface 18
9.3 Database (phpMyAdmin) 21
10. Explanation of PHP OOP Implementation **23**
10.1 OOP Concepts Demonstrated 23
10.2 UML Class Diagram 24
11. Explanation of API / Fetch Communication **25**
11.1 API Endpoints 25
11.2 Slot-Checking Flow (Main Fetch Operation) 25
12. Payment Workflow **28**
12.1 Steps 28
12.2 Test Cards Used 28
12.3 Payment Flowchart and Screens 29
13. Testing **31**
13.1 Test Cases 31
13.2 Test Screenshots 33
13.3 Summary of Results 36
14. Conclusion **37**
14.1 Limitations 37
14.2 Future Work 37
References **38**

IFT302

List of Figures

Figure 5.1: Use-case diagram of the Vehicle Service Booking System 6
Figure 6.1: System architecture diagram 8
Figure 7.1: Entity-relationship diagram of the ui_vsb_db database 9
Figure 9.1: Registration page 13
Figure 9.2: Login page 13
Figure 9.3: Customer dashboard 14
Figure 9.4: Vehicle registration page with vehicle list 14
Figure 9.5: Services page showing prices 15
Figure 9.6: Booking page with available slots loaded using the Fetch API 16
Figure 9.7: Payment page with test-mode banner 17
Figure 9.8: Payment confirmation and receipt with booking reference 17
Figure 9.9: Vehicle tracking page with progress stepper 18
Figure 9.10: Admin dashboard 18
Figure 9.11: Manage services (add/edit modal) 19
Figure 9.12: Bookings list with approve/reject and status update 19
Figure 9.13: Customers page 20
Figure 9.14: Payment records page 20
Figure 9.15: Search service records 21
Figure 9.16: List of tables in the database (phpMyAdmin) 21
Figure 9.17: Bookings table with real stored rows, showing that data is stored in MySQL 22
Figure 9.18: Payments table showing both successful and failed rows 22
Figure 10.1: UML class diagram of the PHP classes 24
Figure 11.1: Fetch request/response sequence (Browser JS → GET /api/slots.php → PHP endpoint → Booking class → MySQL → JSON response → render slot buttons) 26
Figure 11.2: Browser DevTools (Network tab) showing the slots.php request and its JSON response 27
Figure 12.1: Payment workflow flowchart 29
Figure 12.2: Failed payment: card declined message 30
Figure 12.3: Successful payment confirmation with transaction reference 30
Figure 13.1: Successful booking with booking summary and total (T11) 33
Figure 13.2: Slot-full state: all bays shown as Full (T12) 34
Figure 13.3: Declined payment (T14) 34
Figure 13.4 (a): Customer typing an admin URL directly (T22) 35
Figure 13.4 (b): Customer redirected to their own dashboard instead (T22) 35
Figure 13.5: Failed login with error message (T05) 36

IFT302

List of Tables

Table 4.1: Customer functional requirements 4
Table 4.2: Administrator functional requirements 4
Table 4.3: Software requirements 5
Table 8.1: Structure of the users table 10
Table 8.2: Structure of the vehicles table 10
Table 8.3: Structure of the services table 10
Table 8.4: Structure of the bookings table 11
Table 8.5: Structure of the payments table 11
Table 8.6: Structure of the status_history table 12
Table 10.1: PHP classes and their roles 23
Table 11.1: API endpoints 25
Table 12.1: Test cards used by the simulated gateway 28
Table 13.1: Test cases and results 31

IFT302

1. Introduction

The University of Ibadan Maintenance Directorate runs a vehicle workshop that services and repairs university vehicles. Appointments have so far been arranged by visiting the workshop or calling, which is slow and hard to track.

This project is the **Vehicle Service Booking and Payment Platform**, a web application that lets staff register online, add their vehicles, choose a service, check available appointment times, book, pay a service fee through a simulated payment system, and follow the progress of their vehicle. Workshop administrators manage services, prices, bookings, payments and service status from an admin panel.

The system is built with HTML, CSS and JavaScript on the front end, PHP (object-oriented) on the back end, and MySQL for storage. Asynchronous communication using the JavaScript Fetch API is used for slot checking, booking and payment.

IFT302

2. Problem Statement

The current manual process has these problems:

- Customers must travel to the workshop or phone to book, which wastes time.
- There is no live view of free appointment slots, so bookings clash or customers wait.
- Payment is handled manually, and records are easy to lose or dispute.
- Customers cannot see whether their vehicle has been received, is being worked on, or is ready.
- Paper records make it hard for the workshop to search past services or see customer and payment history.

A web-based system is needed to automate booking, payment and tracking, and to store the records in a database.

IFT302

3. Objectives

Main objective: To design and implement a web-based vehicle service booking and payment platform for the University of Ibadan Maintenance Directorate.

Specific objectives:

1. Allow customers to register, log in and manage their vehicle details.
2. Display available services with their prices.
3. Check available appointment slots in real time using the Fetch API.
4. Allow customers to submit a booking and receive a unique booking reference.
5. Provide a simulated payment workflow with a payment confirmation.
6. Let customers track the status of their vehicle after submission.
7. Give administrators tools to manage services and prices, approve or reject bookings, update service status, and view customer and payment records.
8. Store and retrieve all data using a MySQL database.

IFT302

4. System Requirements

4.1 Functional Requirements

Table 4.1: Customer functional requirements

ID	Requirement
FR1	Register an account and log in/out.
FR2	Add, view and delete own vehicles (reg number, make, model, year).
FR3	View services and their costs.
FR4	Select a vehicle, service and preferred date.
FR5	Check available slots asynchronously.
FR6	Submit a booking and receive a booking reference.
FR7	Pay via simulated payment and receive confirmation.
FR8	Track booking and vehicle service progress.

Table 4.2: Administrator functional requirements

ID	Requirement
FR9	Add, edit, delete services and set prices.
FR10	View all bookings and search/filter them.
FR11	Approve or reject bookings.
FR12	Update vehicle service status.
FR13	View customer information.
FR14	View payment records.
FR15	Search service records.

4.2 Non-Functional Requirements

- **Security:** hashed passwords, prepared statements, role-based access, session protection.
- **Usability:** simple, responsive interface that works on desktop and mobile browsers.
- **Performance:** slot availability returned in under a second on a local server.
- **Reliability:** transactions prevent two users booking the last slot at the same time.
- **Maintainability:** modular OOP code, one class per domain concept.
- **Payment safety:** only simulated payments; full card numbers and CVV are never stored.

IFT302

4.3 Software Requirements

Table 4.3: Software requirements

Item	**Used**
Front end	HTML5, CSS3, JavaScript (ES6, Fetch API)
Back end	PHP 8.x
Database	MySQL / MariaDB
Server	Apache via XAMPP/WAMP
Tools	VS Code, phpMyAdmin, Git, browser (Chrome/Edge)

4.4 Hardware Requirements

Any computer with at least 4 GB RAM, 2 GHz processor and 1 GB free disk space; a network connection for team collaboration.

IFT302

5. Use-Case Diagram

Actors: Customer, Workshop Administrator, Payment Gateway (simulated, secondary actor).

Customer use cases: Register, Login, Manage Vehicles, View Services, Check Available Slots, Book Service, Make Payment, View Confirmation, Track Vehicle Status.

Admin use cases: Login, Manage Services, Set Prices, View Bookings, Approve/Reject Booking, Update Service Status, View Customers, View Payments, Search Records.

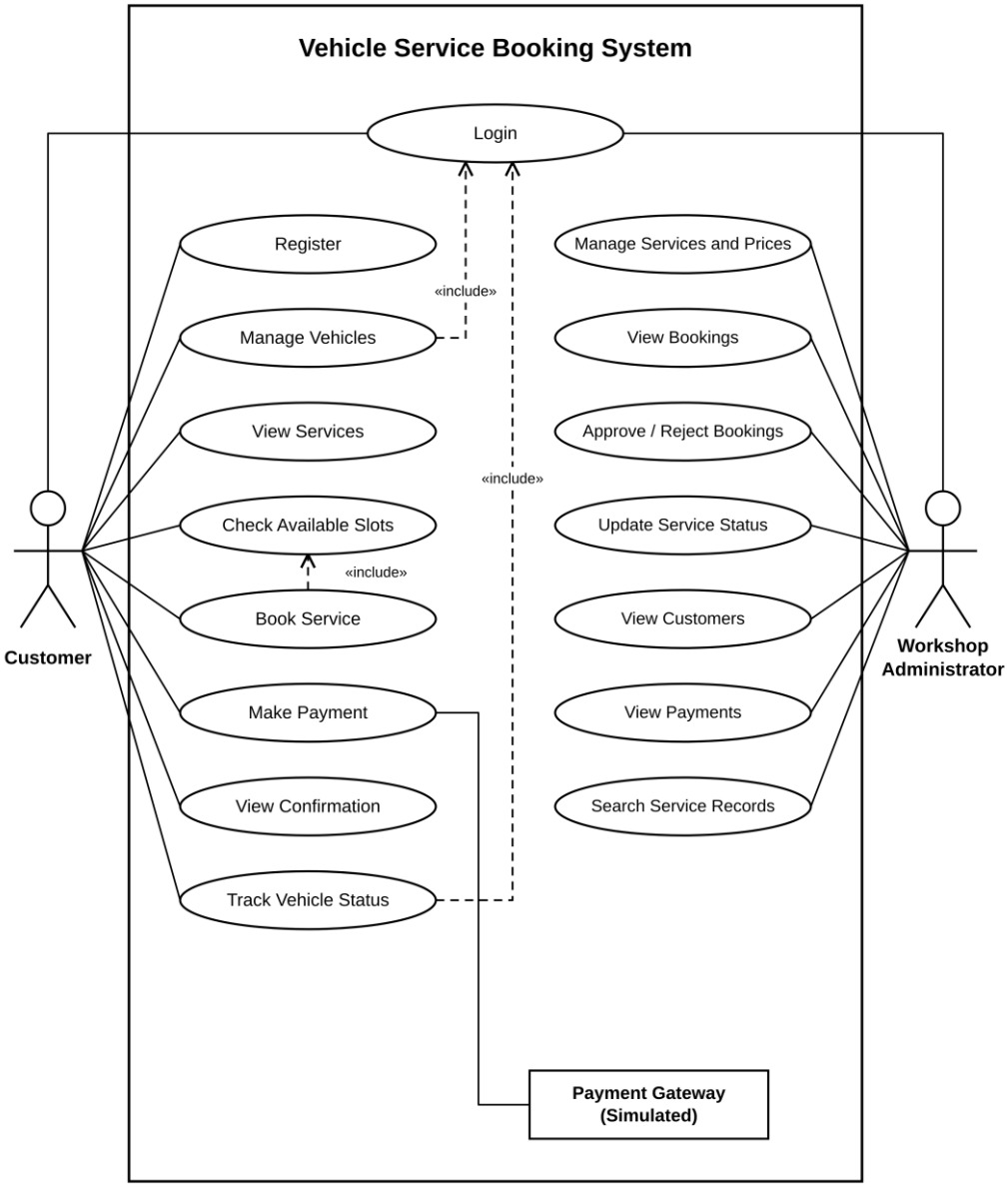


Figure 5.1: Use-case diagram of the Vehicle Service Booking System

6. System Architecture

The system uses a **three-tier (client–server) architecture**:

1. **Presentation layer:** HTML, CSS and JavaScript in the browser. Pages are rendered by PHP; dynamic actions (slots, booking, payment, admin updates) use `fetch()` to call the API.
2. **Application layer:** PHP on Apache. API endpoint files handle requests and delegate to OOP classes (User, Vehicle, Service, Booking, Payment, Auth, SimulatedPaymentGateway).
3. **Data layer:** MySQL database accessed through PDO prepared statements.

IFT302

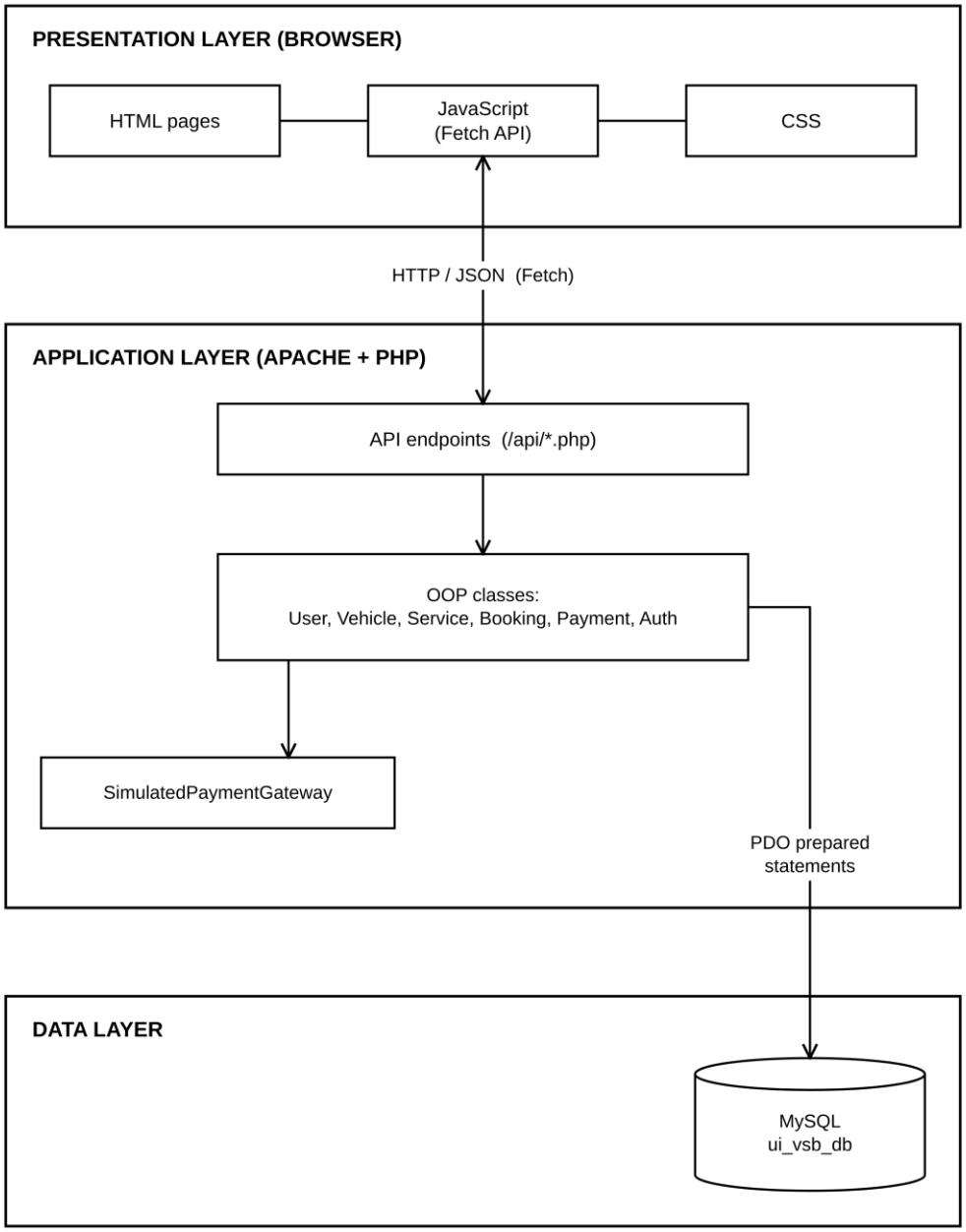


Figure 6.1: System architecture diagram

7. Database / ER Diagram

The database `ui_vsb_db` has six tables: `users`, `vehicles`, `services`, `bookings`, `payments` and `status_history`. A user (customer) owns many vehicles and makes many bookings. Each booking is for one vehicle and one service, can have payment attempts, and has a history of status changes.

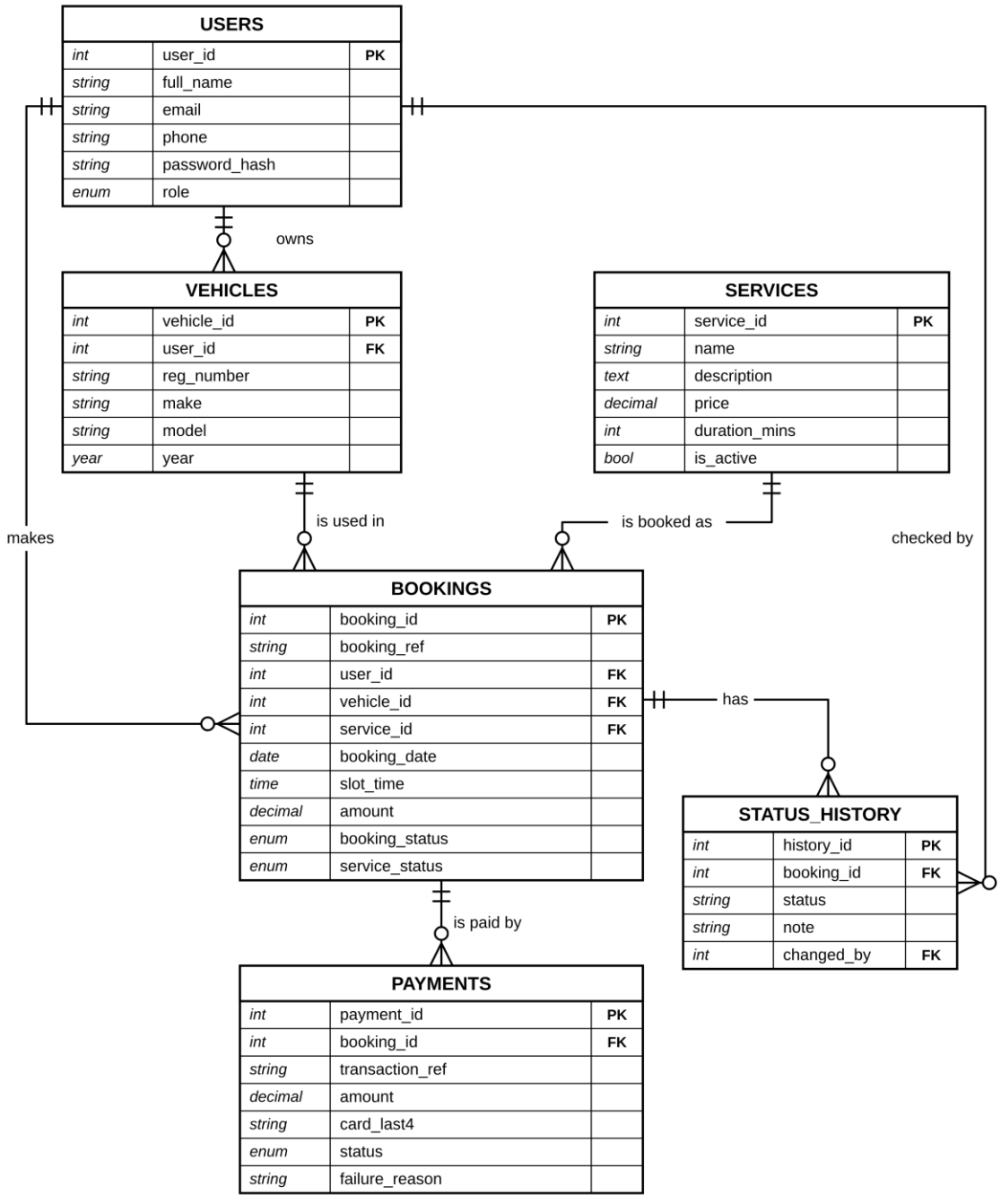


Figure 7.1: Entity-relationship diagram of the ui_vsb_db database

8. Database Tables

The structure of each of the six tables in the `ui_vsb_db` database is given below.

Table 8.1: Structure of the users table

Column	Type	Constraints	Description
`user_id`	INT	PK, AUTO_INCREMENT	Unique user id
`full_name`	VARCHAR(100)	NOT NULL	Full name
`email`	VARCHAR(120)	UNIQUE, NOT NULL	Login email
`phone`	VARCHAR(20)	NOT NULL	Phone number
`password_hash`	VARCHAR(255)	NOT NULL	Hashed password
`role`	ENUM('customer','admin')	NOT NULL	User role
`created_at`	TIMESTAMP	DEFAULT NOW	Registration time

Table 8.2: Structure of the vehicles table

Column	Type	Constraints	Description
`vehicle_id`	INT	PK, AUTO_INCREMENT	Unique vehicle id
`user_id`	INT	FK → users	Owner
`reg_number`	VARCHAR(20)	UNIQUE, NOT NULL	Registration number
`make`	VARCHAR(50)	NOT NULL	Manufacturer
`model`	VARCHAR(50)	NOT NULL	Model
`year`	YEAR	NOT NULL	Year of manufacture
`created_at`	TIMESTAMP	DEFAULT NOW	Date added

Table 8.3: Structure of the services table

Column	Type	Constraints	Description
`service_id`	INT	PK, AUTO_INCREMENT	Unique service id
`name`	VARCHAR(100)	NOT NULL	Service name
`description`	TEXT		What the service includes
`price`	DECIMAL(10,2)	NOT NULL	Fee in naira
`duration_mins`	INT	NOT NULL	Estimated duration
`is_active`	TINYINT(1)	DEFAULT 1	Shown to customers if 1

IFT302

Column	Type	Constraints	Description
created_at	TIMESTAMP	DEFAULT NOW	Created time

Table 8.4: Structure of the bookings table

Column	Type	Constraints	Description
booking_id	INT	PK, AUTO_INCREMENT	Unique id
booking_ref	VARCHAR(20)	UNIQUE, NOT NULL	e.g. VSB-20261006-A4K9
user_id	INT	FK → users	Customer
vehicle_id	INT	FK → vehicles	Vehicle
service_id	INT	FK → services	Service
booking_date	DATE	NOT NULL	Appointment date
slot_time	TIME	NOT NULL	Appointment time
amount	DECIMAL(10,2)	NOT NULL	Price at booking time
booking_status	ENUM	NOT NULL	pending_payment, pending_approval, approved, rejected, cancelled
service_status	ENUM	NOT NULL	not_started, received, in_progress, completed, ready_for_pickup
notes	VARCHAR(255)	NULL	Optional notes
created_at	TIMESTAMP	DEFAULT NOW	Created time

Table 8.5: Structure of the payments table

Column	Type	Constraints	Description
payment_id	INT	PK, AUTO_INCREMENT	Unique id
booking_id	INT	FK → bookings	Booking paid for
transaction_ref	VARCHAR(30)	UNIQUE, NOT NULL	e.g. TXN-20261006-8F3K2A
amount	DECIMAL(10,2)	NOT NULL	Amount charged
method	VARCHAR(20)	DEFAULT 'test_card'	Payment method
card_last4	CHAR(4)	NULL	Last four card digits only
status	ENUM('success','failed','refunded')	NOT NULL	Outcome
failure_reason	VARCHAR(100)	NULL	Reason if failed
paid_at	TIMESTAMP	DEFAULT NOW	Time of attempt

IFT302

Table 8.6: Structure of the status_history table

Column	Type	Constraints	Description
`history_id`	INT	PK, AUTO_INCREMENT	Unique id
`booking_id`	INT	FK → bookings	Related booking
`status`	VARCHAR(30)	NOT NULL	Status recorded
`note`	VARCHAR(255)	NULL	Optional note
`changed_by`	INT	FK → users	Who made the change
`changed_at`	TIMESTAMP	DEFAULT NOW	When

IFT302

9. Important Interface Screenshots

9.1 Customer Interface

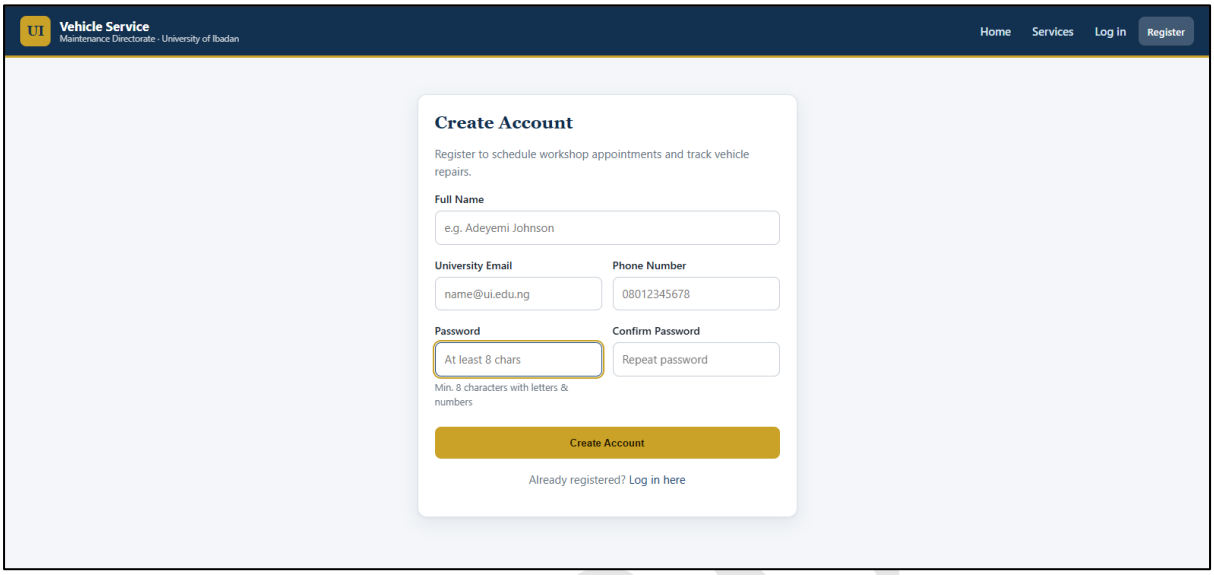


Figure 9.1: Registration page

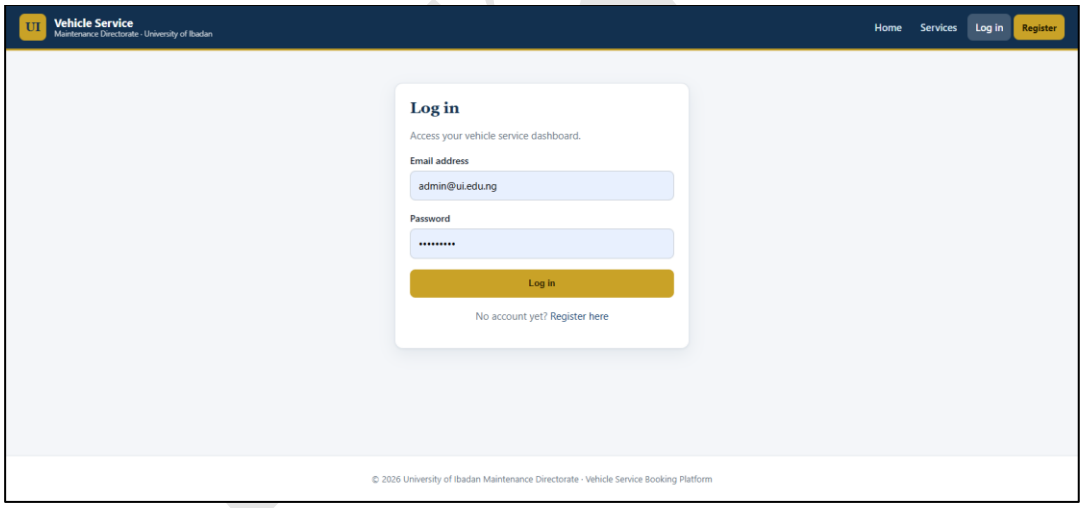


Figure 9.2: Login page

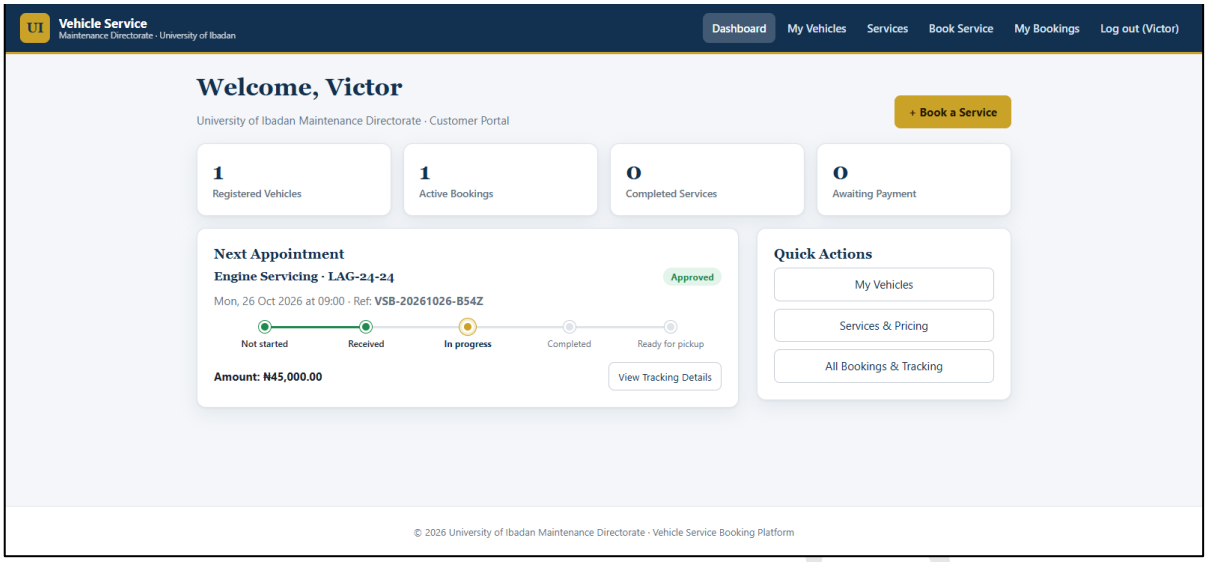


Figure 9.3: Customer dashboard

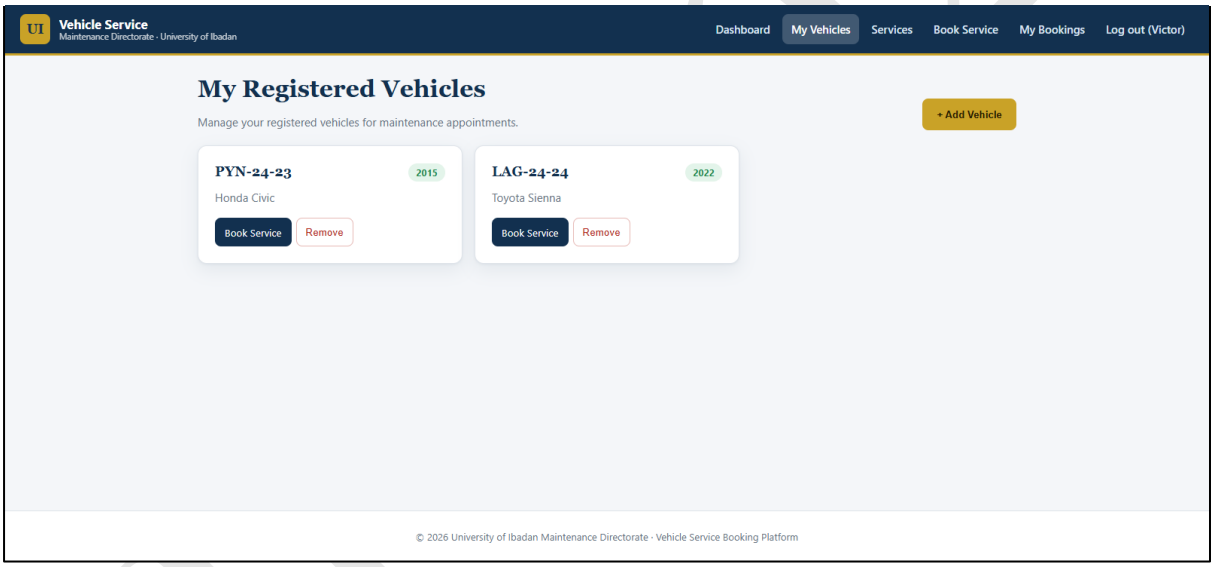


Figure 9.4: Vehicle registration page with vehicle list

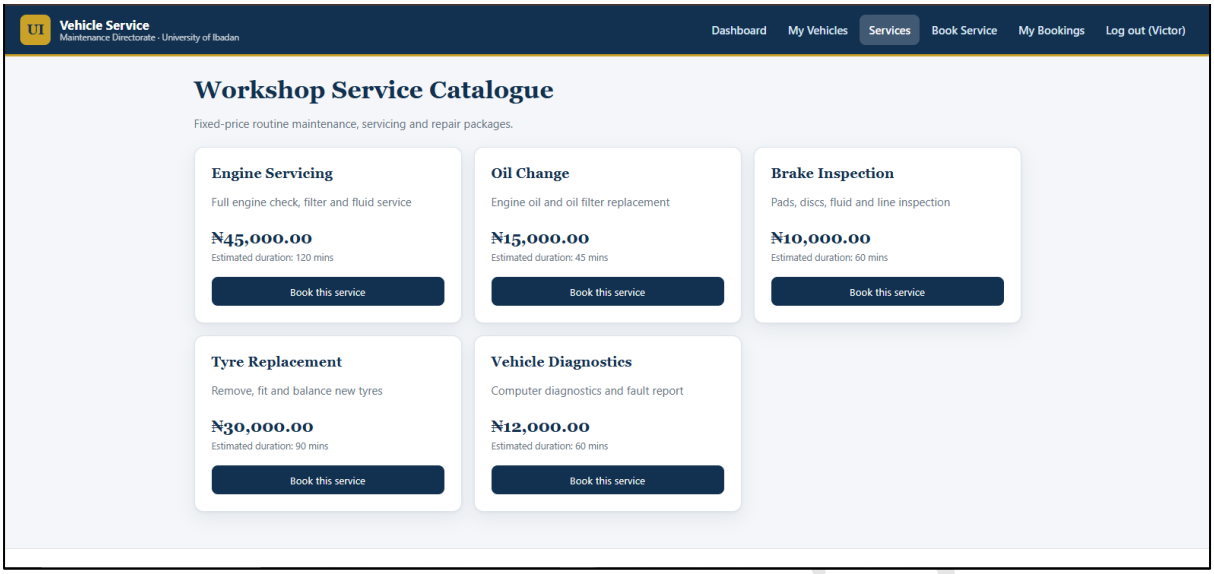


Figure 9.5: Services page showing prices

IFT302

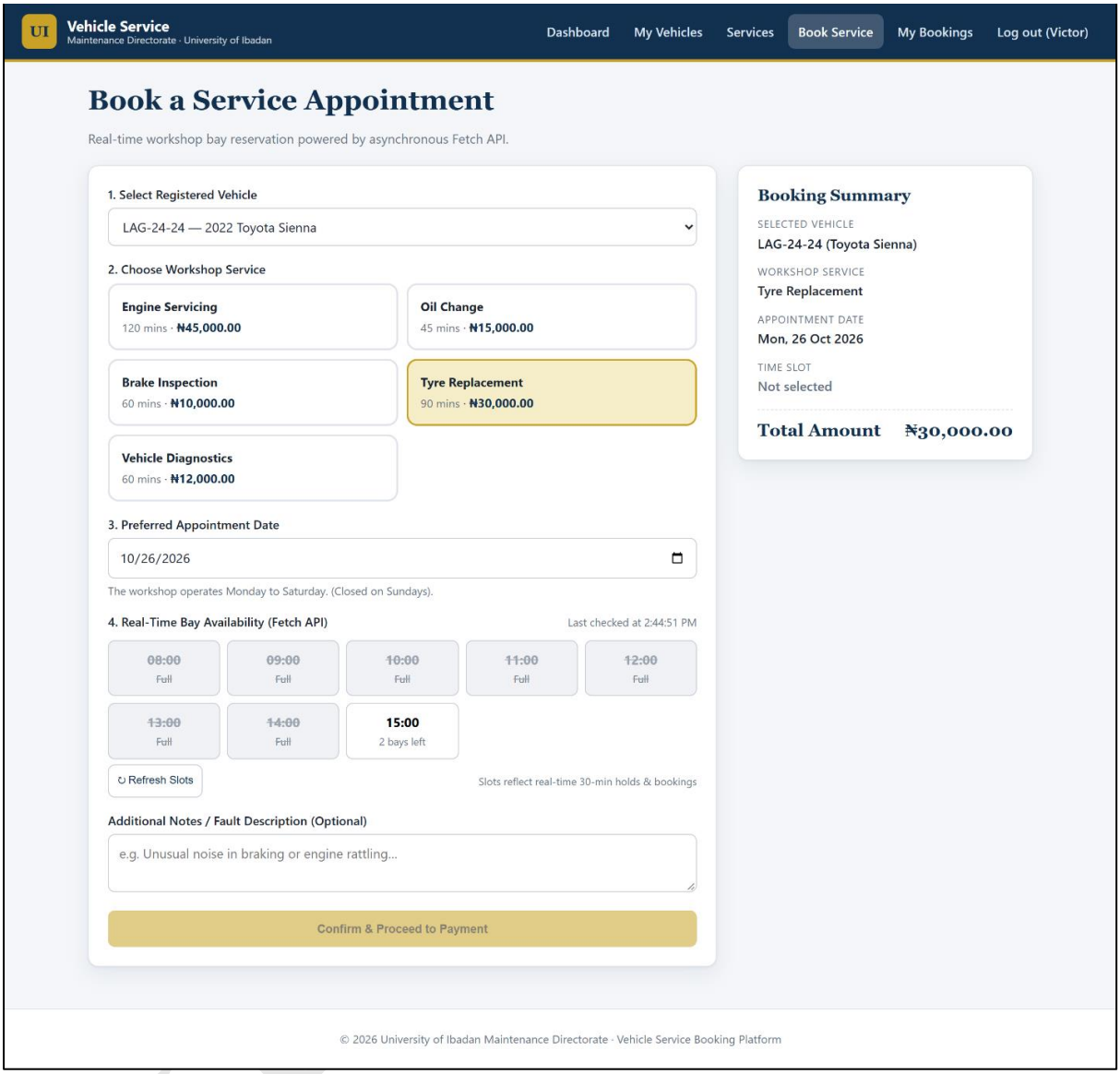


Figure 9.6: Booking page with available slots loaded using the Fetch API

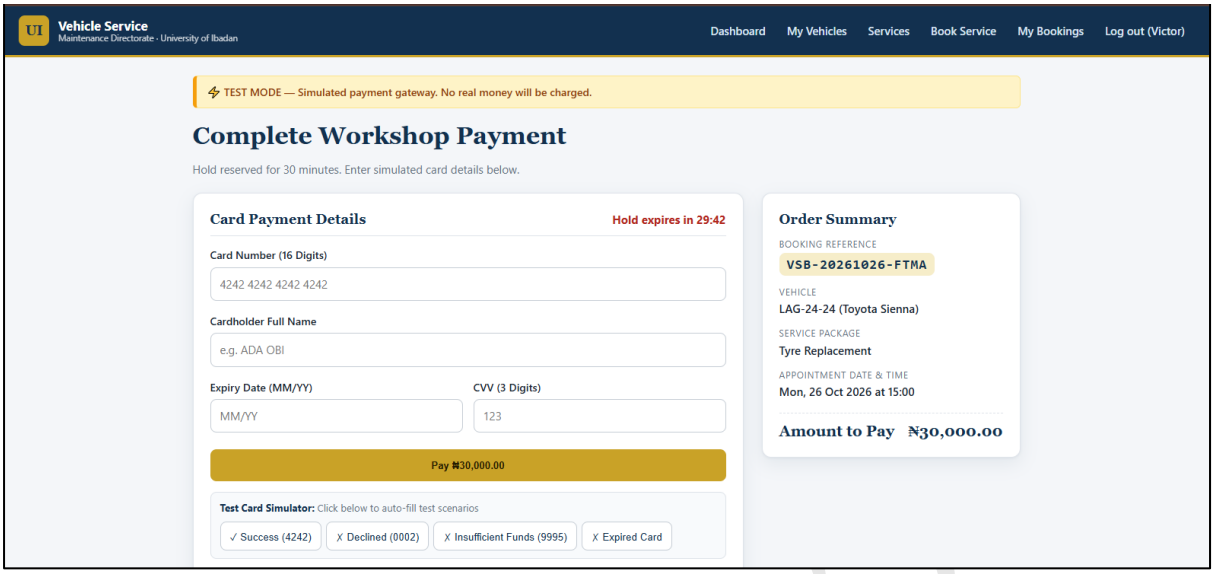


Figure 9.7: Payment page with test-mode banner

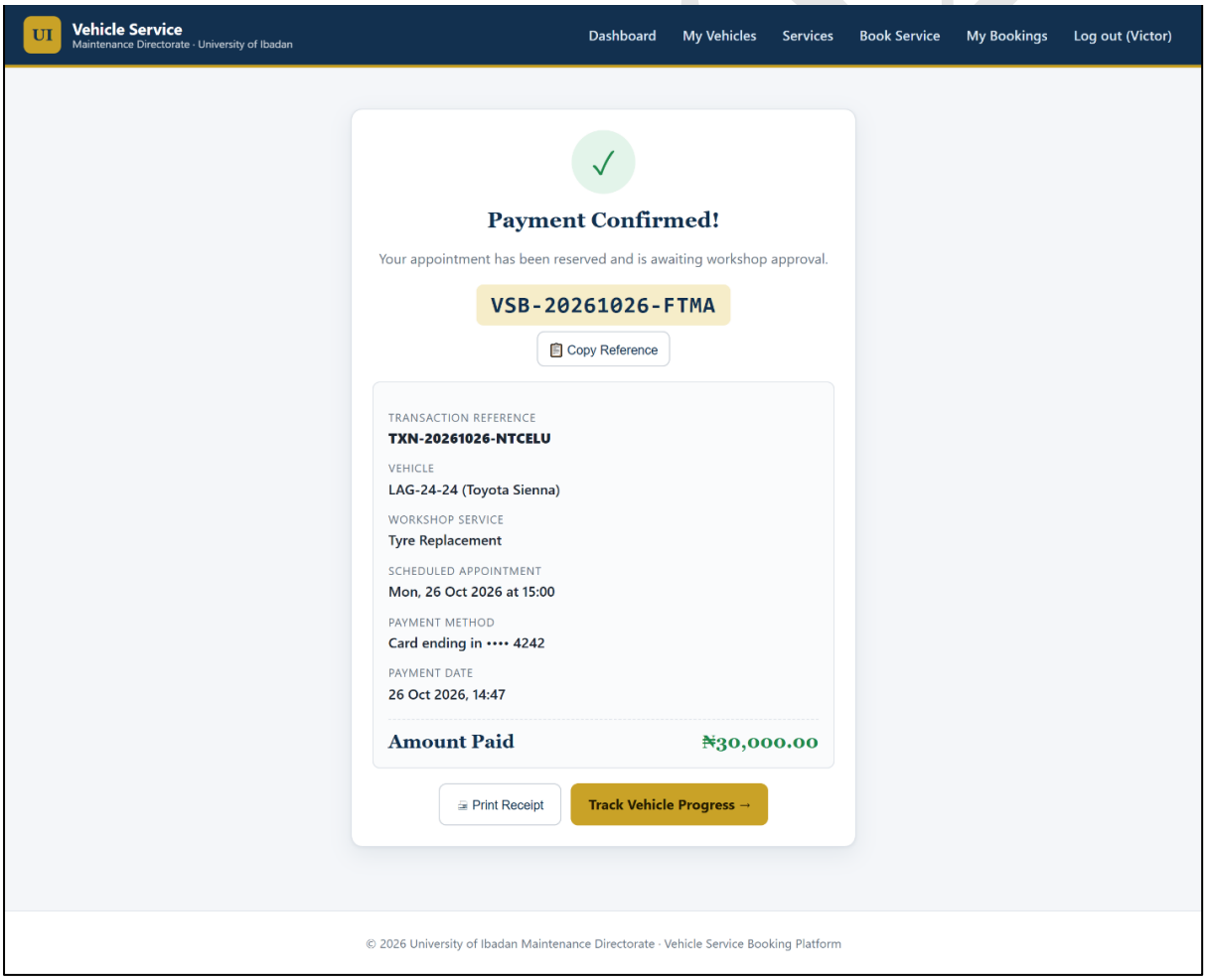


Figure 9.8: Payment confirmation and receipt with booking reference

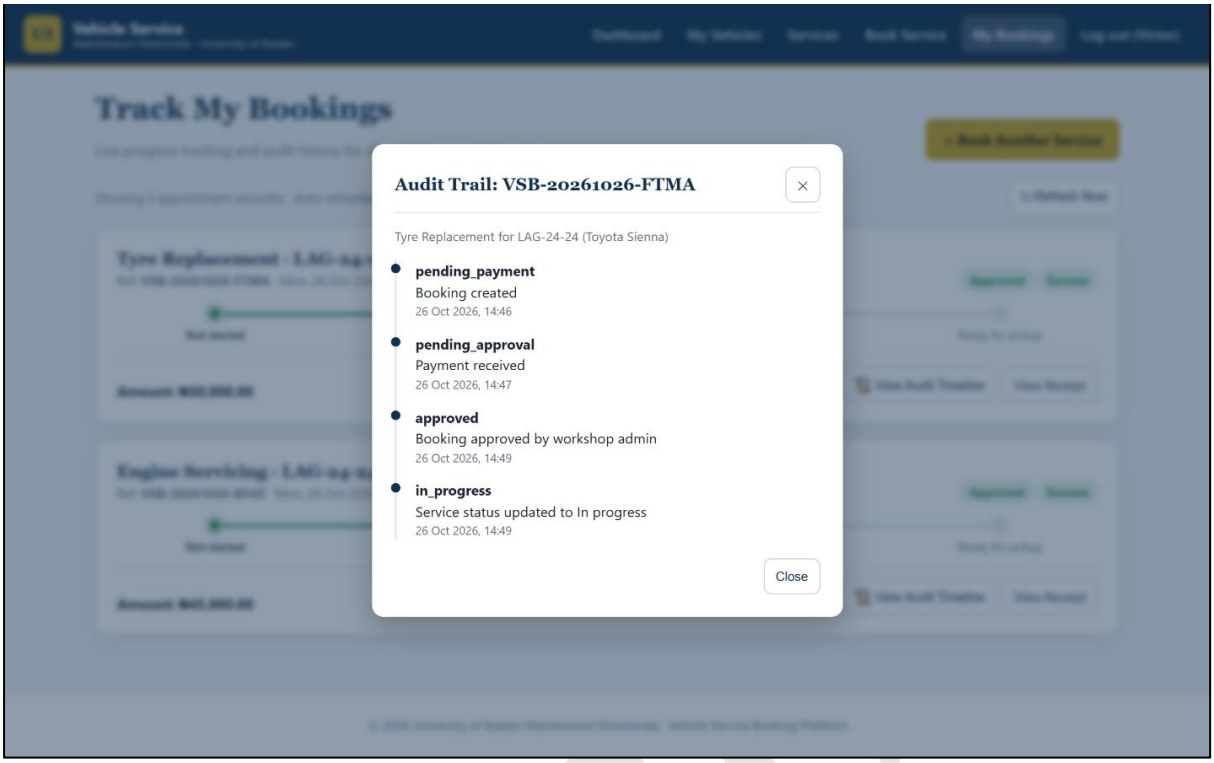


Figure 9.9: Vehicle tracking page with progress stepper

9.2 Administrator Interface

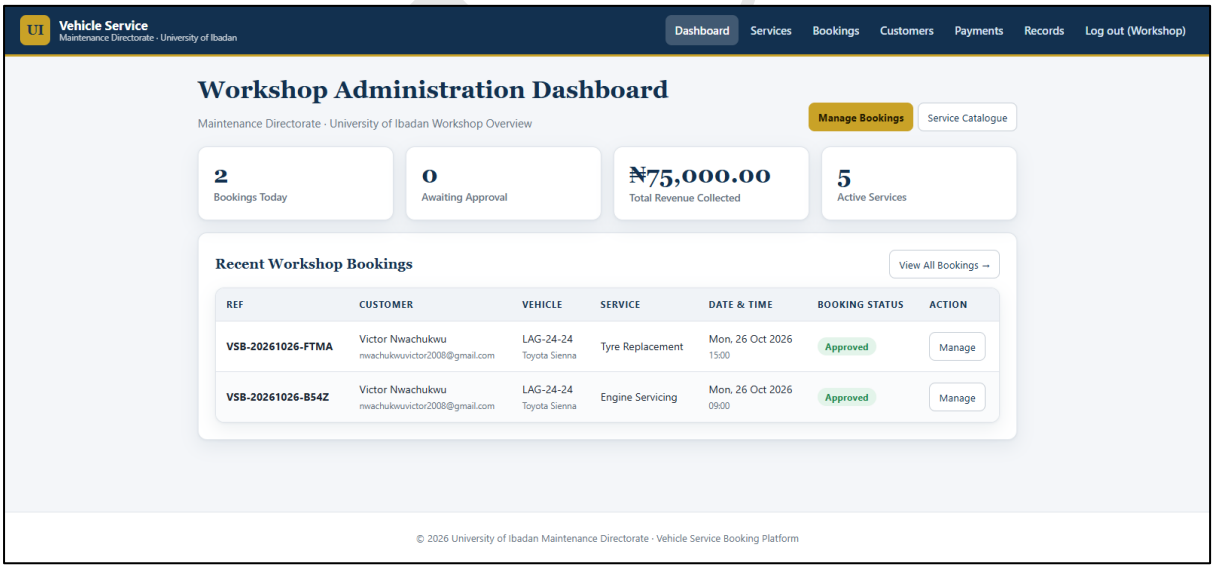


Figure 9.10: Admin dashboard

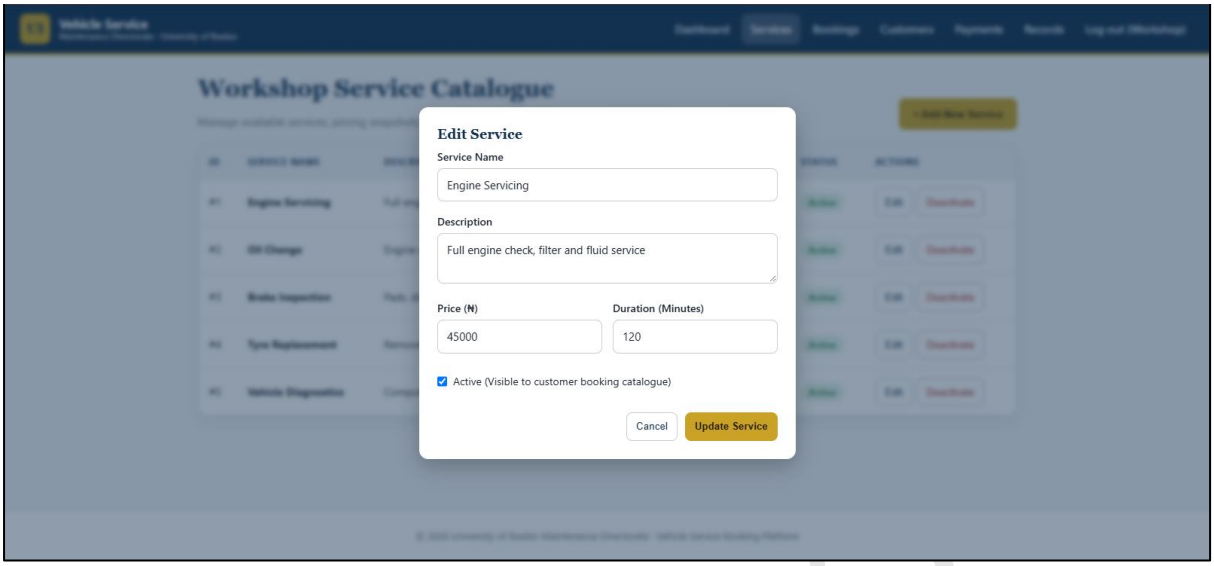


Figure 9.11: Manage services (add/edit modal)

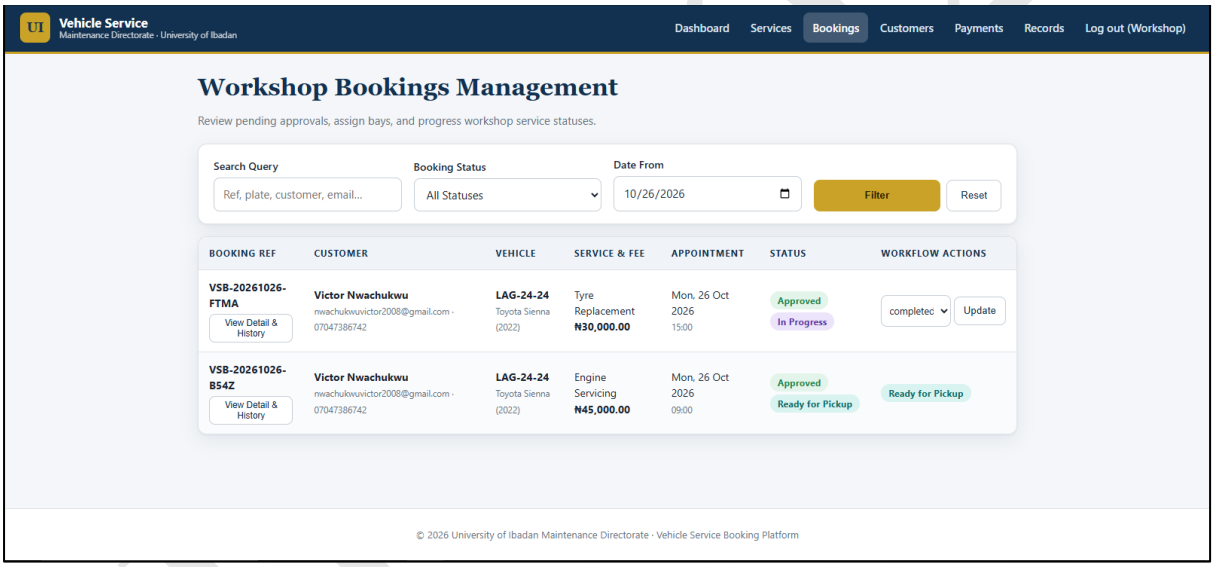


Figure 9.12: Bookings list with approve/reject and status update

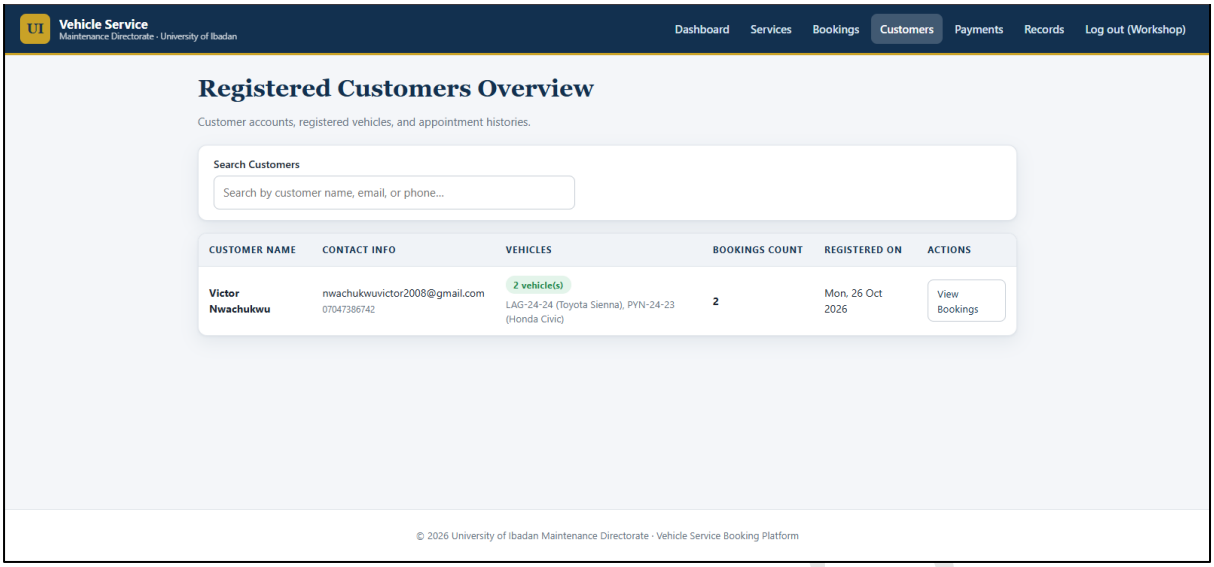


Figure 9.13: Customers page

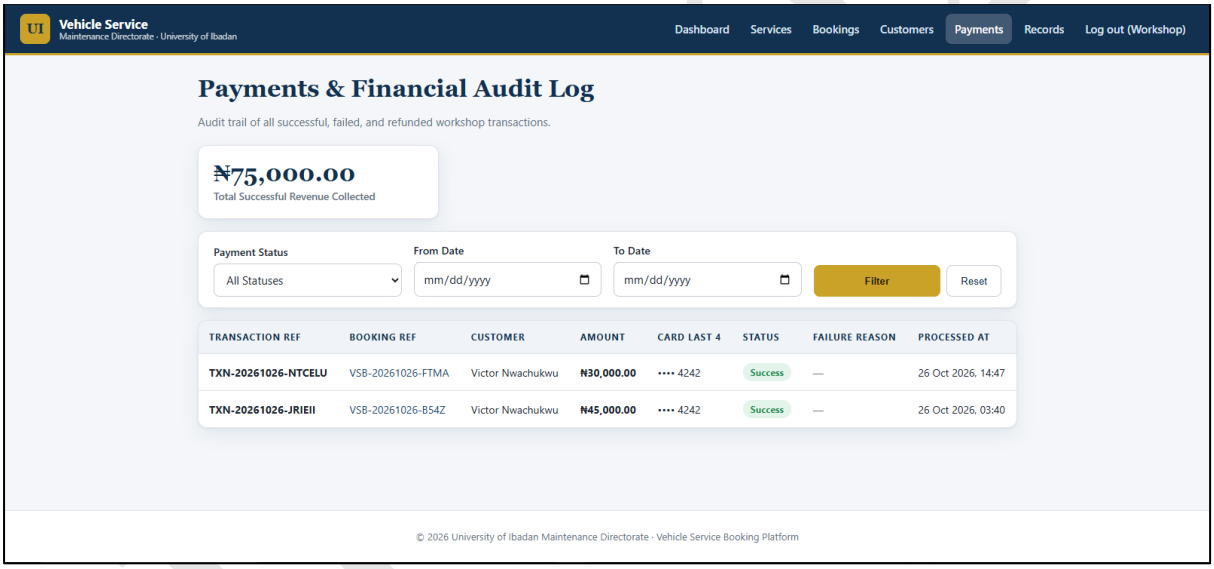


Figure 9.14: Payment records page

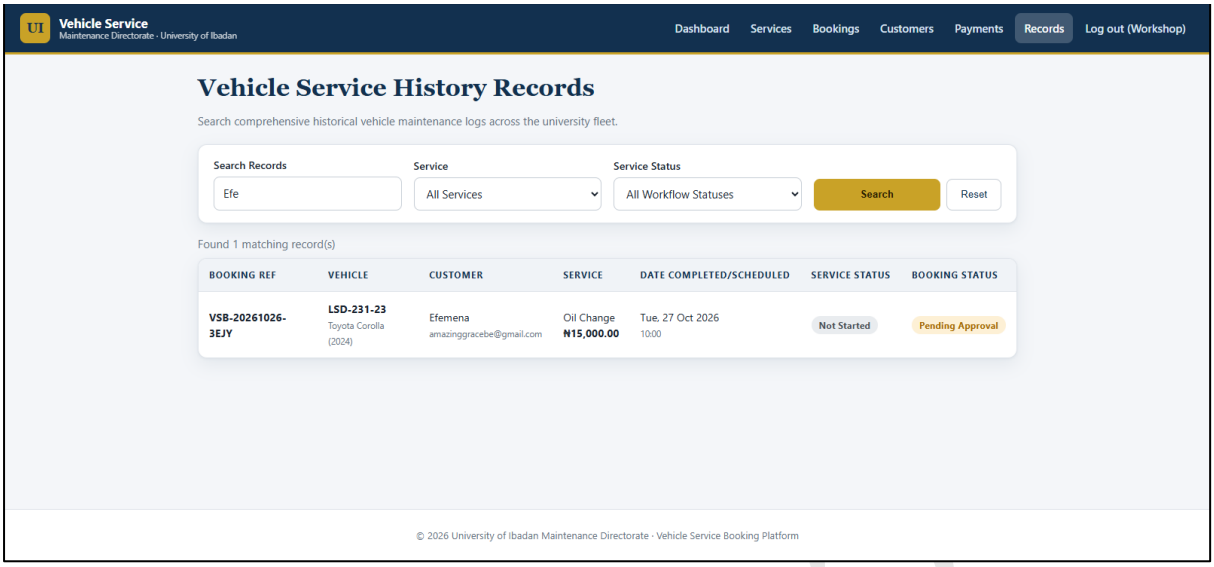


Figure 9.15: Search service records

9.3 Database (phpMyAdmin)

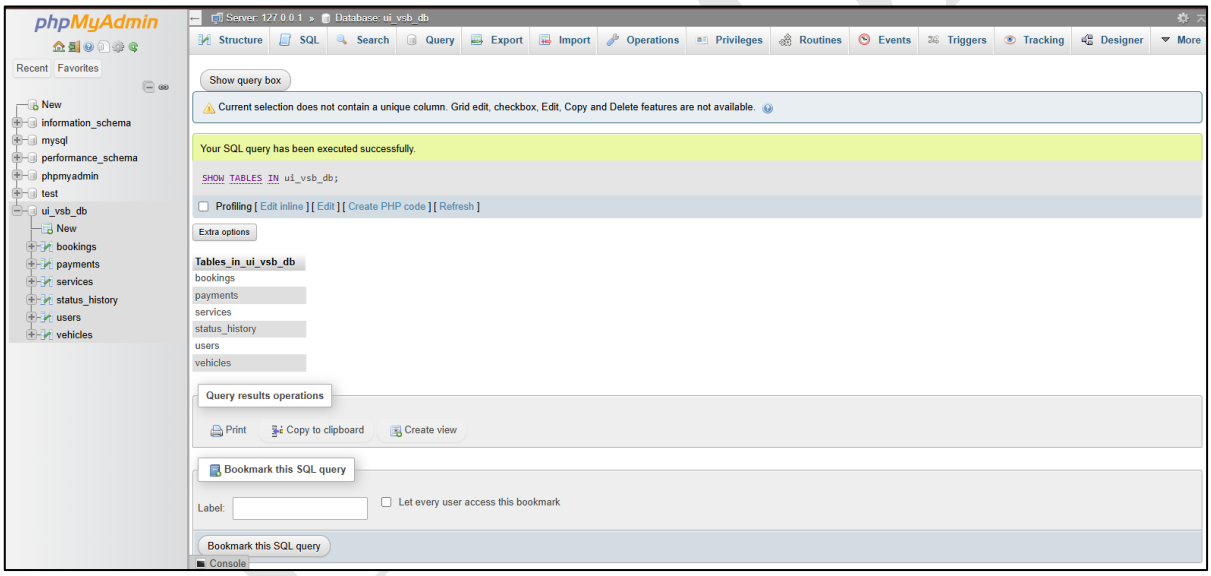


Figure 9.16: List of tables in the database (phpMyAdmin)

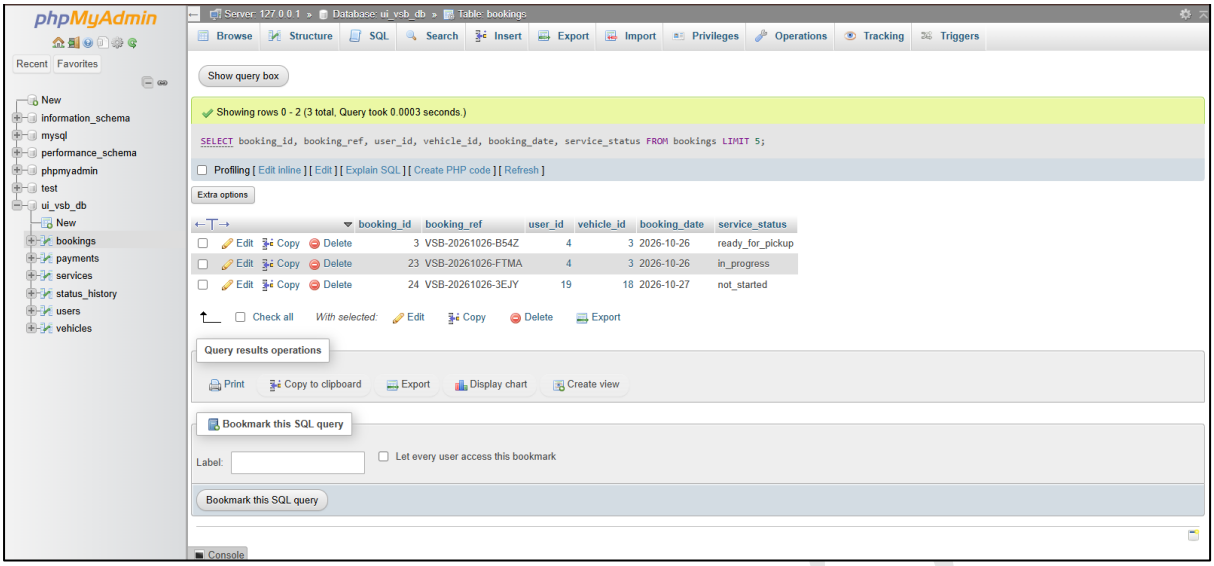


Figure 9.17: Bookings table with real stored rows, showing that data is stored in MySQL

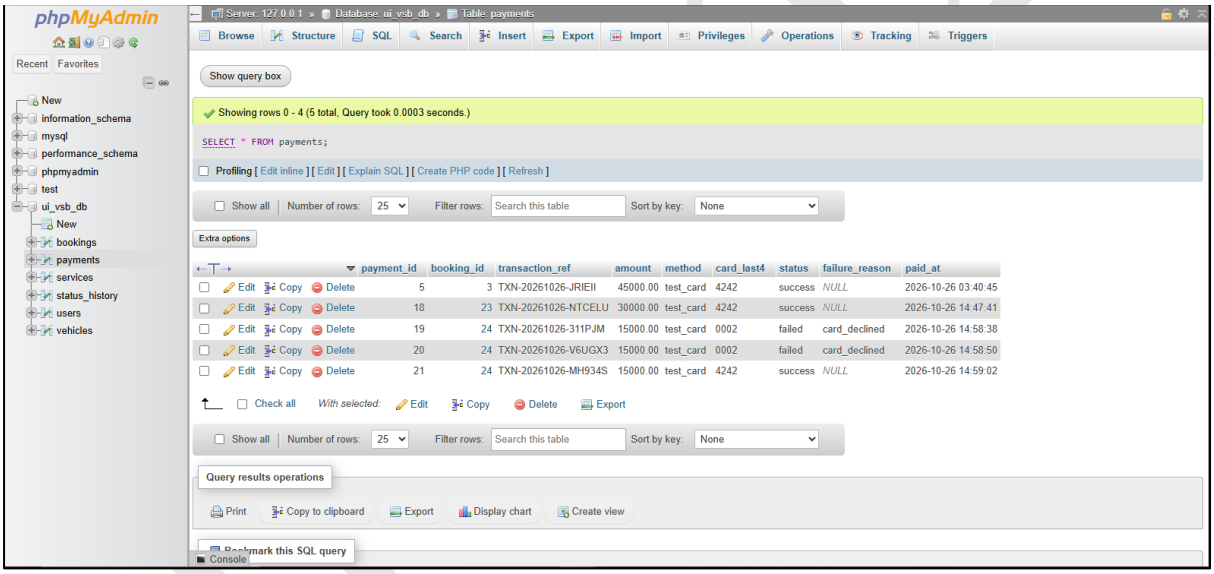


Figure 9.18: Payments table showing both successful and failed rows

10. Explanation of PHP OOP Implementation

The back end is organised into classes in the `src/` folder, loaded with an autoloader (`spl_autoload_register`). Each class represents one concept of the problem domain and hides its database logic behind methods.

Table 10.1: PHP classes and their roles

Class	Role
`Database`	Singleton that creates one PDO connection shared across the app.
`User`	Registers users (hashing passwords), logs them in, lists customers for the admin.
`Vehicle`	Adds, lists, checks ownership of and deletes vehicles.
`Service`	Retrieves, creates, updates and deletes services and prices.
`Booking`	Generates slots, checks availability, creates bookings in a transaction, approves/rejects, updates service status, writes history.
`Payment`	Records, retrieves and refunds payments.
`PaymentGatewayInterface`	Declares charge(), the contract every gateway must follow.
`SimulatedPaymentGateway`	Implements the interface using test-card rules; no real money moves.
`Auth`	Starts sessions, protects pages and API endpoints by role.
`ApiResponse`	Produces consistent JSON responses with correct HTTP codes.

10.1 OOP Concepts Demonstrated

- **Encapsulation:** database connection and internal logic are private; other code only uses public methods.
- **Abstraction and interface:** `PaymentGatewayInterface` lets the payment code depend on a contract, not a specific gateway.
- **Polymorphism:** `SimulatedPaymentGateway` can be replaced by a real gateway class without changing the Payment flow.
- **Singleton pattern:** `Database` ensures a single connection.
- **Separation of concerns:** API files only receive requests and return JSON; classes do the business logic and data access.

IFT302

10.2 UML Class Diagram

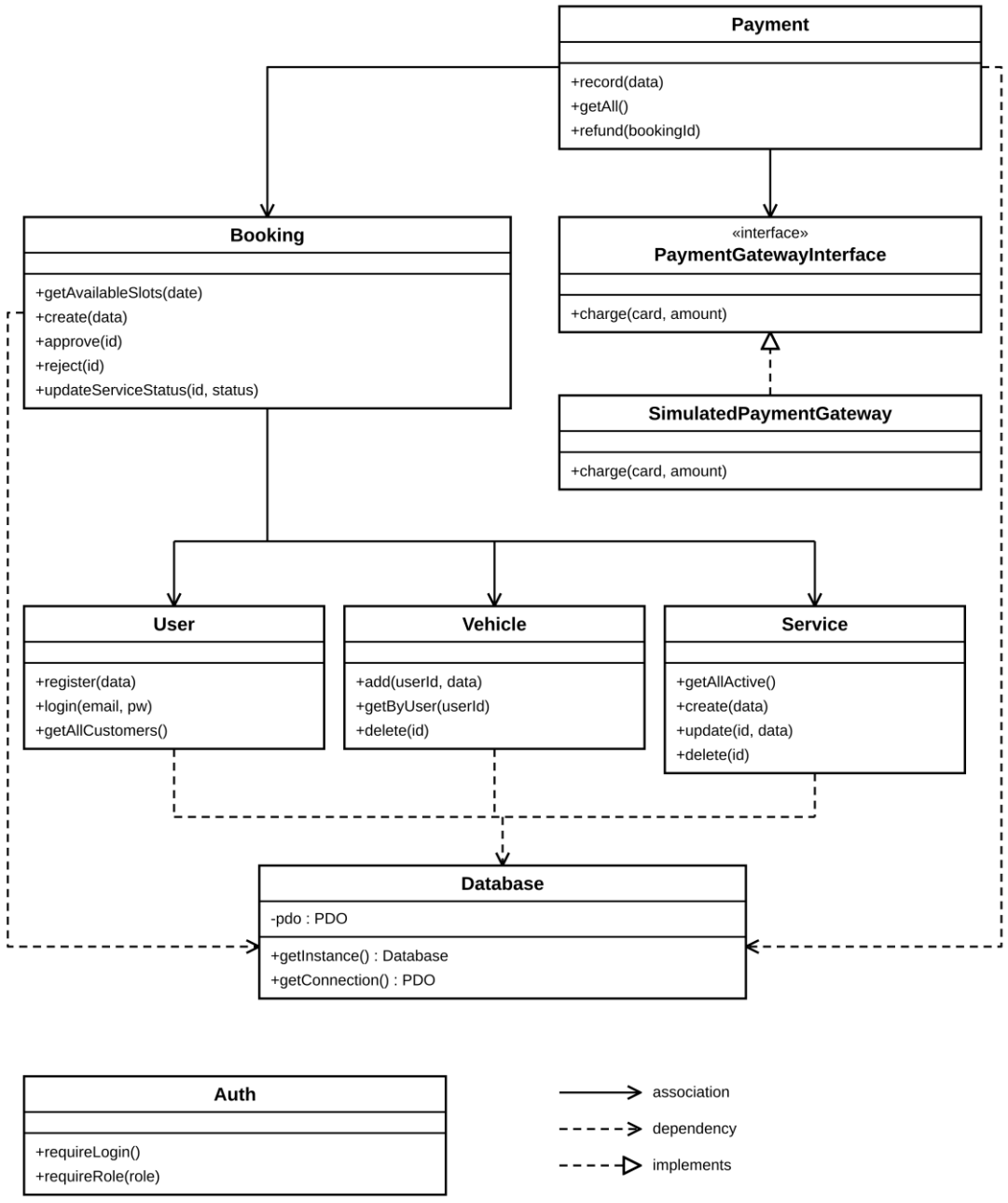


Figure 10.1: UML class diagram of the PHP classes

11. Explanation of API / Fetch Communication

The Fetch API (`fetch()` in JavaScript) lets a page send a request to the server in the background and receive JSON back without reloading the page. In ui-vsb, three major operations use it: slot checking, booking submission and payment. Admin actions such as approving or rejecting a booking and updating service status also use it.

11.1 API Endpoints

Table 11.1: API endpoints

Method	Endpoint	Purpose
GET	`/api/slots.php?date=&service_id=`	Returns available slots for a date
POST	`/api/book.php`	Creates a booking and returns a booking reference
POST	`/api/payment.php`	Processes the simulated payment
GET	`/api/booking_status.php`	Returns booking and service progress
GET/POST/DELETE	`/api/vehicles.php`	Manages customer vehicles
POST/PUT/DELETE	`/api/admin/services.php`	Admin service and price management
GET/POST	`/api/admin/bookings.php`	Admin list, search, approve/reject
POST	`/api/admin/status.php`	Admin updates service status
GET	`/api/admin/customers.php`	Admin views customers
GET	`/api/admin/payments.php`	Admin views payments

Every endpoint returns JSON in one consistent shape, `{ "success": true|false, "message": "...", "data": ... }`, together with a suitable HTTP status code (200, 201, 400, 401, 403, 409 or 422). The `ApiResponse` class produces this shape, so the front end can handle every response in the same way.

11.2 Slot-Checking Flow (Main Fetch Operation)

1. The customer selects a service and a date on the booking page.
2. JavaScript sends `fetch('/api/slots.php?date=2026-10-12&service_id=3')`.
3. The PHP endpoint calls `Booking::getAvailableSlots()`, which counts the existing bookings for each time slot in MySQL.
4. The server returns JSON listing each slot as available or full.
5. JavaScript builds the slot buttons on the page with no reload; full slots are disabled.

Example JSON response:

```
{ "success": true, "message": "Slots loaded",
  "data": [ {"time":"08:00","available":true,"remaining":2},
            {"time":"09:00","available":true,"remaining":1},
            {"time":"10:00","available":false,"remaining":0} ] }
```

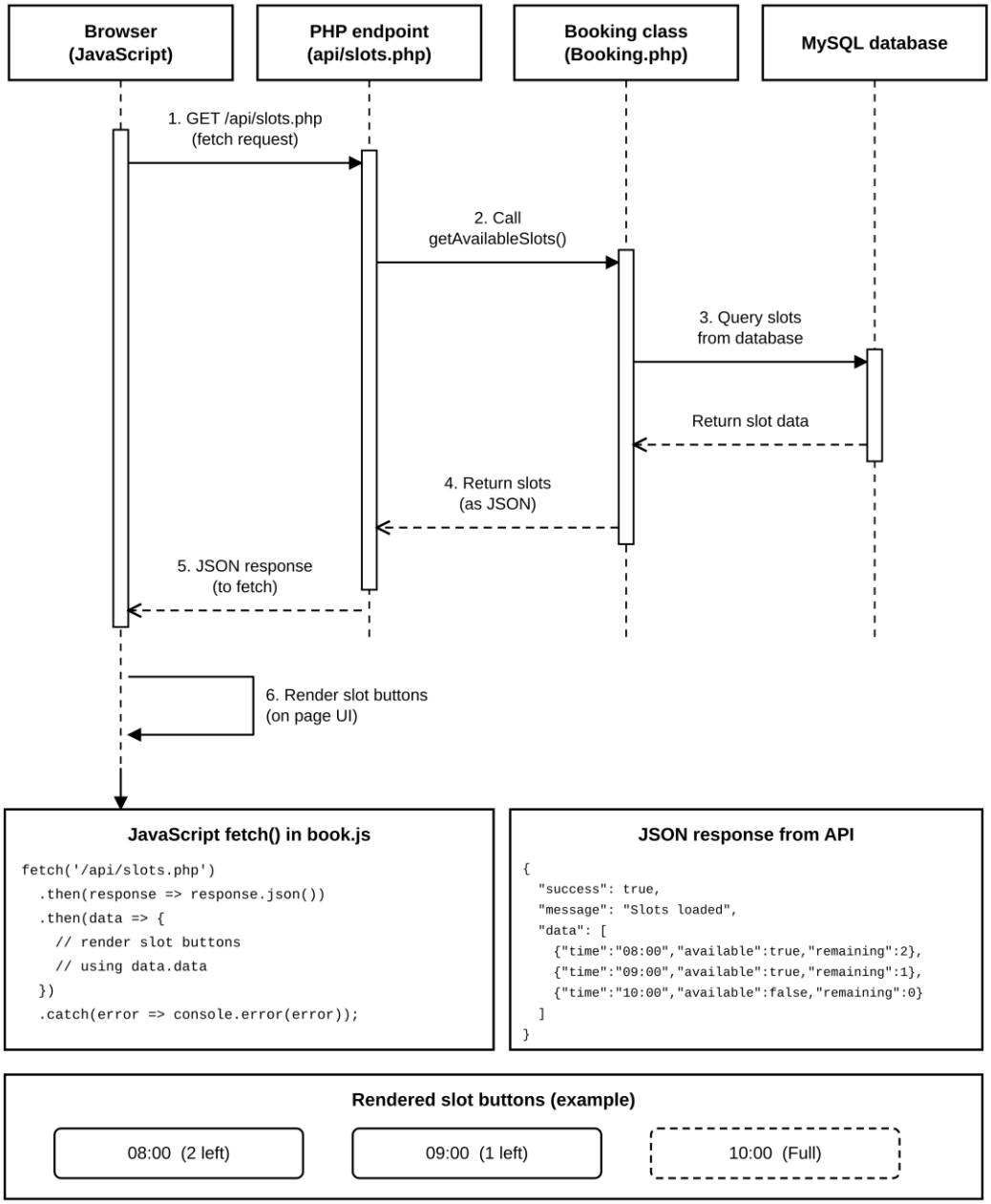


Figure 11.1: Fetch request/response sequence (Browser JS → GET /api/slots.php → PHP endpoint → Booking class → MySQL → JSON response → render slot buttons)

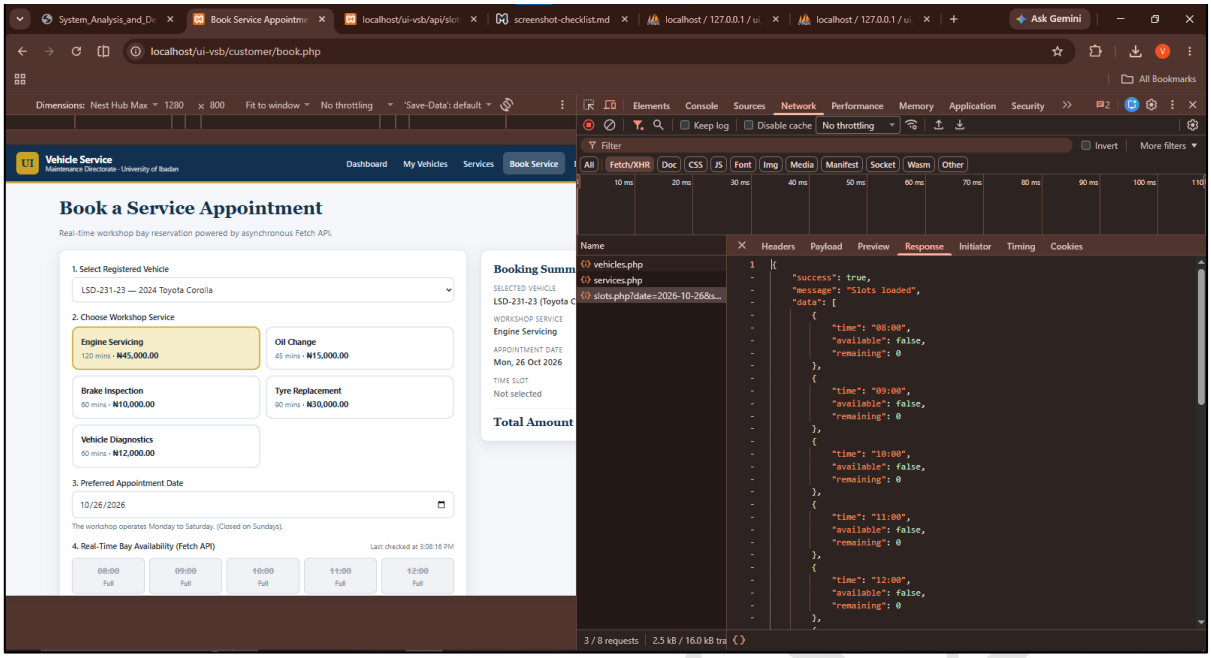


Figure 11.2: Browser DevTools (Network tab) showing the slots.php request and its JSON response

IFT302

12. Payment Workflow

Payment in this system is simulated. No real card is charged and no real bank is contacted. A `SimulatedPaymentGateway` class, which implements the `PaymentGatewayInterface`, mimics a real gateway using test cards. Only the last four digits of a card are ever stored.

12.1 Steps

1. After the booking is created (status `pending_payment`), the customer is taken to the payment page, which shows the order summary and amount.
2. The customer enters test card details. Client-side checks validate the format.
3. JavaScript sends the details to `/api/payment.php` using `fetch()`, and a loading indicator is shown.
4. The server checks that the user owns the booking and that it is unpaid, then validates the card (length, Luhn check, expiry, CVV).
5. The `SimulatedPaymentGateway` returns success or a failure reason.
6. **Success:** a payments row (`success`) is saved with a transaction reference and the last four digits; the booking moves to `pending_approval`; the customer sees a confirmation with the booking reference and receipt.
7. **Failure:** a payments row (`failed`) is saved with the reason; the booking stays unpaid; the customer sees an error and may retry.
8. The administrator later approves the booking, or rejects it, in which case the payment is marked `refunded` (simulated).

12.2 Test Cards Used

Table 12.1: Test cards used by the simulated gateway

Card number	Result
4242 4242 4242 4242	Successful payment
4000 0000 0000 0002	Declined (card_declined)
4000 0000 0000 9995	Declined (insufficient_funds)
Any card with a past expiry date	Declined (expired_card)

IFT302

12.3 Payment Flowchart and Screens

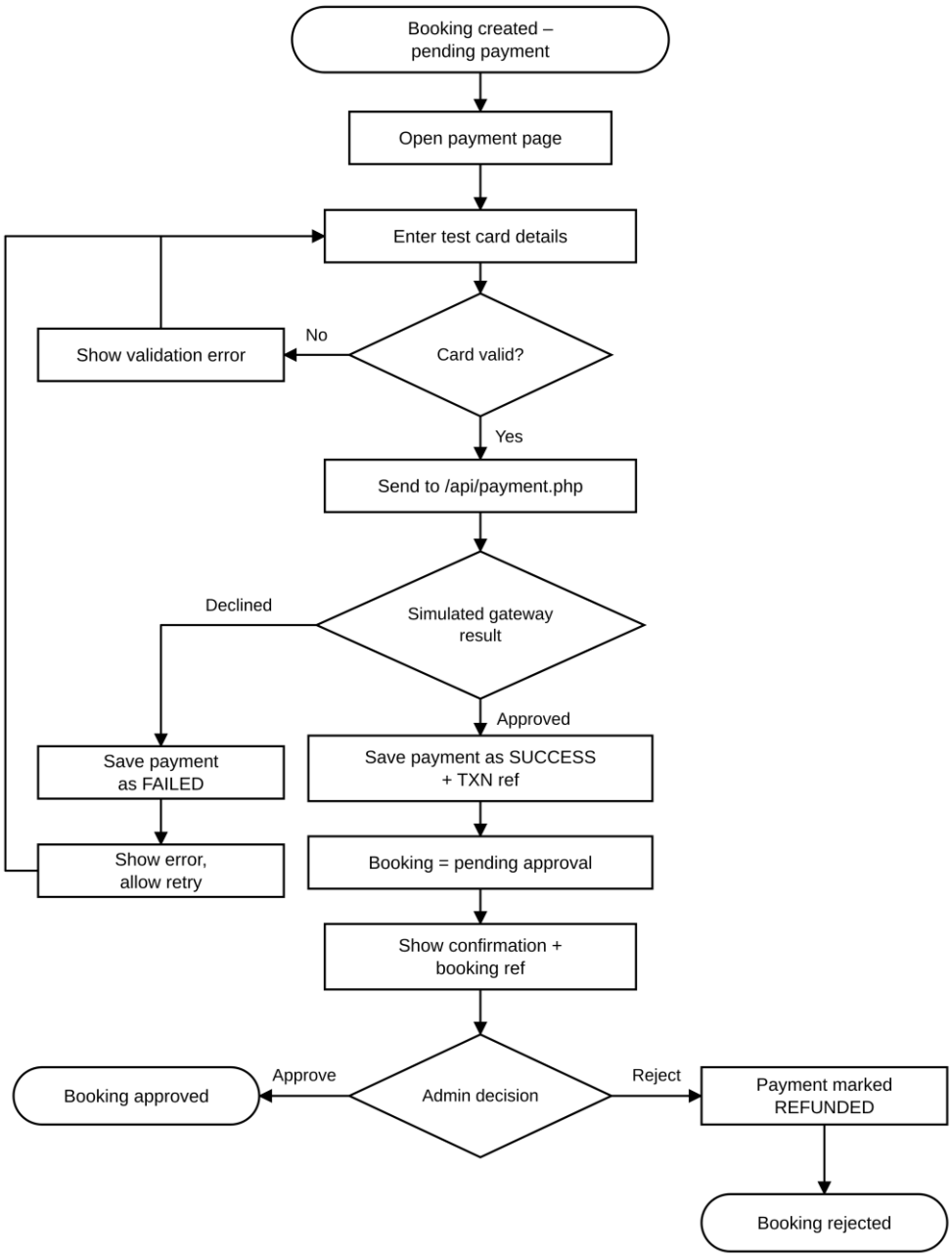


Figure 12.1: Payment workflow flowchart

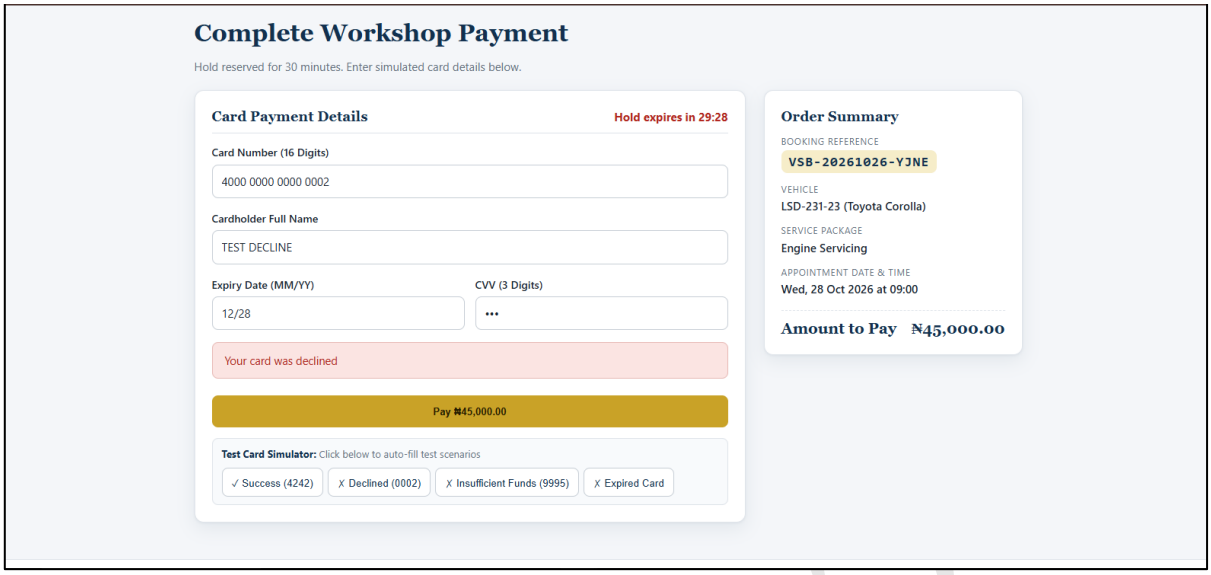


Figure 12.2: Failed payment: card declined message

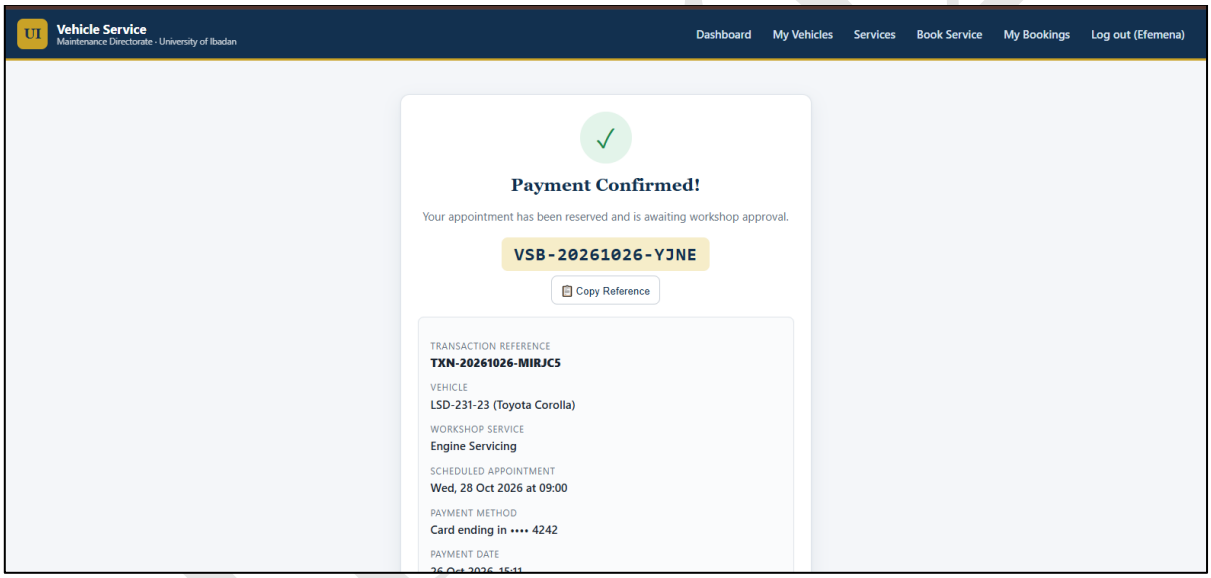


Figure 12.3: Successful payment confirmation with transaction reference

13. Testing

Testing was done manually on a local XAMPP server. Both successful and failed operations were tested. Each test case lists the action taken, the expected result and whether it passed.

13.1 Test Cases

Table 13.1: Test cases and results

ID	Feature	Input / Action	Expected Result	Status
T01	Register	Valid details	Account created	Pass
T02	Register	Email already used	Error: email exists	Pass
T03	Register	Password under 8 characters	Validation error	Pass
T04	Login	Correct credentials	Redirect to dashboard	Pass
T05	Login	Wrong password	Error, no login	Pass
T06	Add vehicle	Valid details	Saved and listed	Pass
T07	Add vehicle	Duplicate registration number	Error shown	Pass
T08	View services	Open services page	Services and prices loaded from DB	Pass
T09	Check slots (Fetch)	Select a date	Slots load without page reload	Pass
T10	Check slots	Date in the past	Rejected	Pass
T11	Book service	Valid selection	Booking reference generated and saved	Pass
T12	Book service	Slot already full	Error 409, slots refresh	Pass
T13	Payment	Card 4242 4242 4242 4242	Success, receipt, status pending approval	Pass
T14	Payment	Card 4000 0000 0000 0002	Declined, booking unpaid, failed row stored	Pass
T15	Payment	Expired card	Declined with reason	Pass
T16	Payment	Invalid card number	Rejected with validation error	Pass
T17	Admin approve	Approve a paid booking	Status becomes approved	Pass
T18	Admin reject	Reject a paid booking	Rejected, payment refunded	Pass
T19	Update status	Set to In progress	Customer tracking page updates	Pass
T20	Service CRUD	Add / edit / delete a service	Database updated	Pass
T21	Search records	Search by registration number	Matching records shown	Pass

IFT302

ID	Feature	Input / Action	Expected Result	Status
T22	Access control	Customer opens an admin URL	Access denied / redirect	Pass
T23	Access control	Logged-out user opens dashboard	Redirect to login	Pass
T24	Security	' OR 1=1 -- typed into login	No bypass (prepared statements)	Pass

IFT302

13.2 Test Screenshots

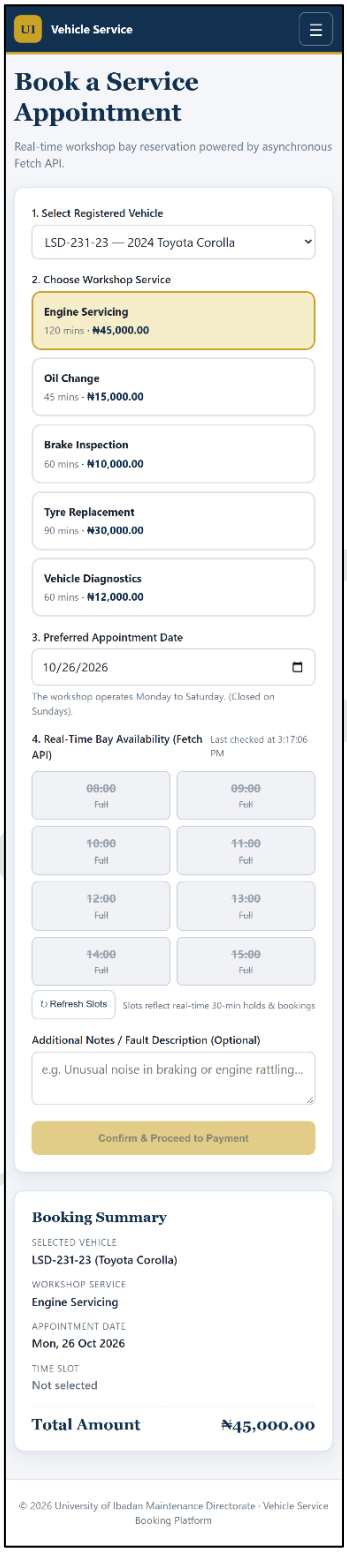


Figure 13.1: Successful booking with booking summary and total (T11)

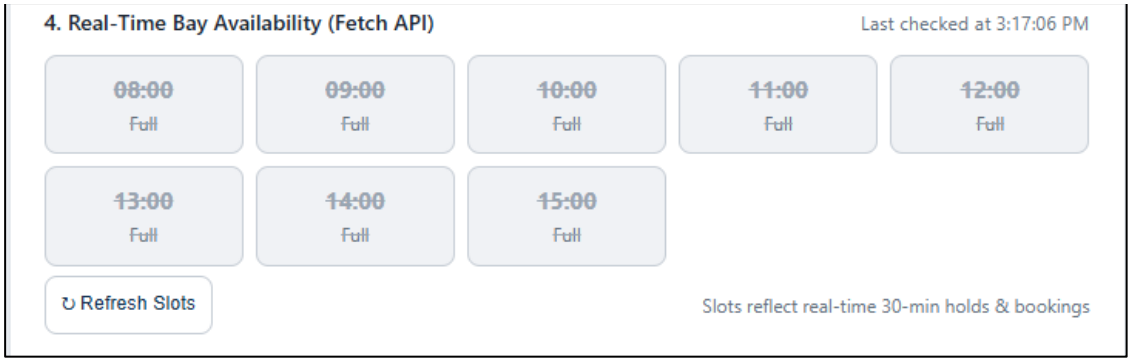


Figure 13.2: Slot-full state: all bays shown as Full (T12)

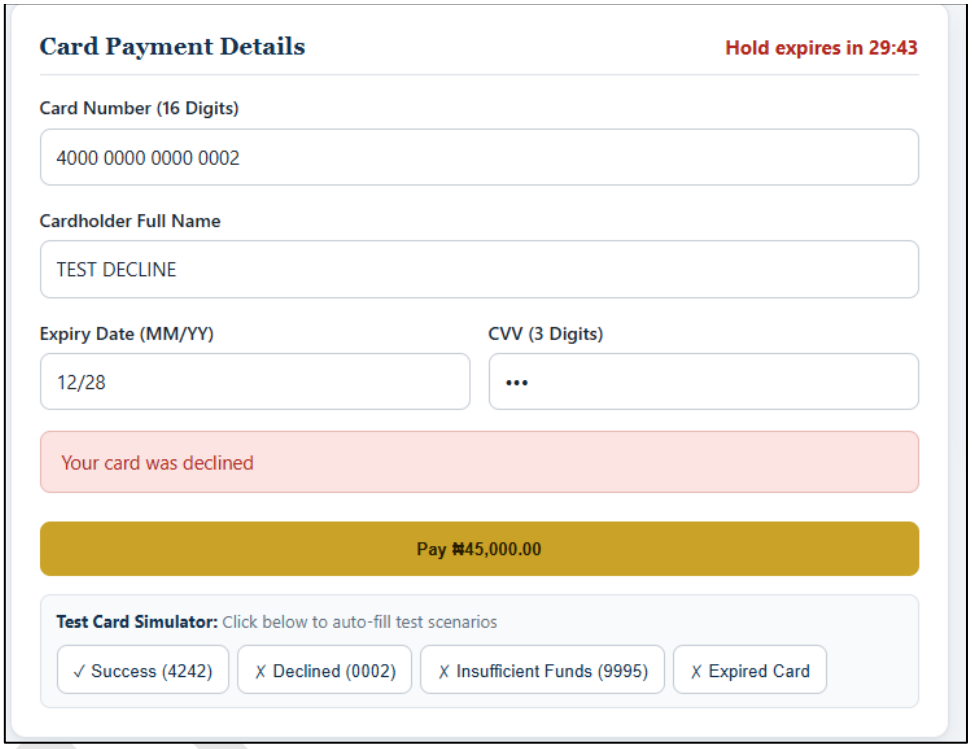


Figure 13.3: Declined payment (T14)

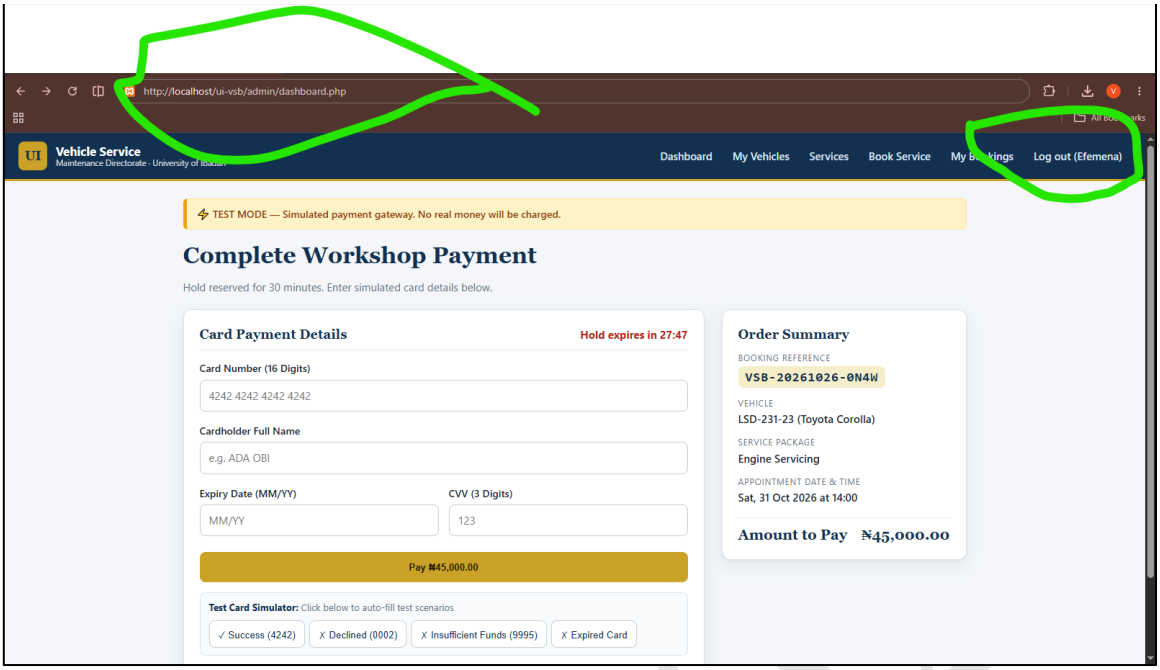


Figure 13.4 (a): Customer typing an admin URL directly (T22)

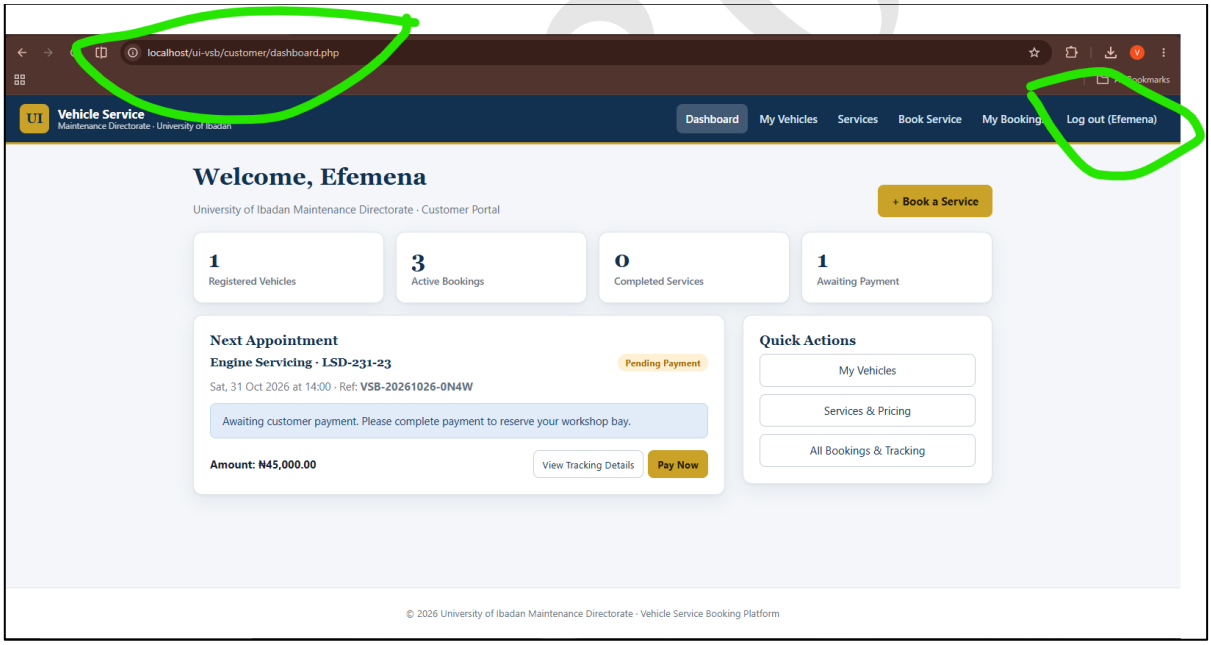


Figure 13.4 (b): Customer redirected to their own dashboard instead (T22)

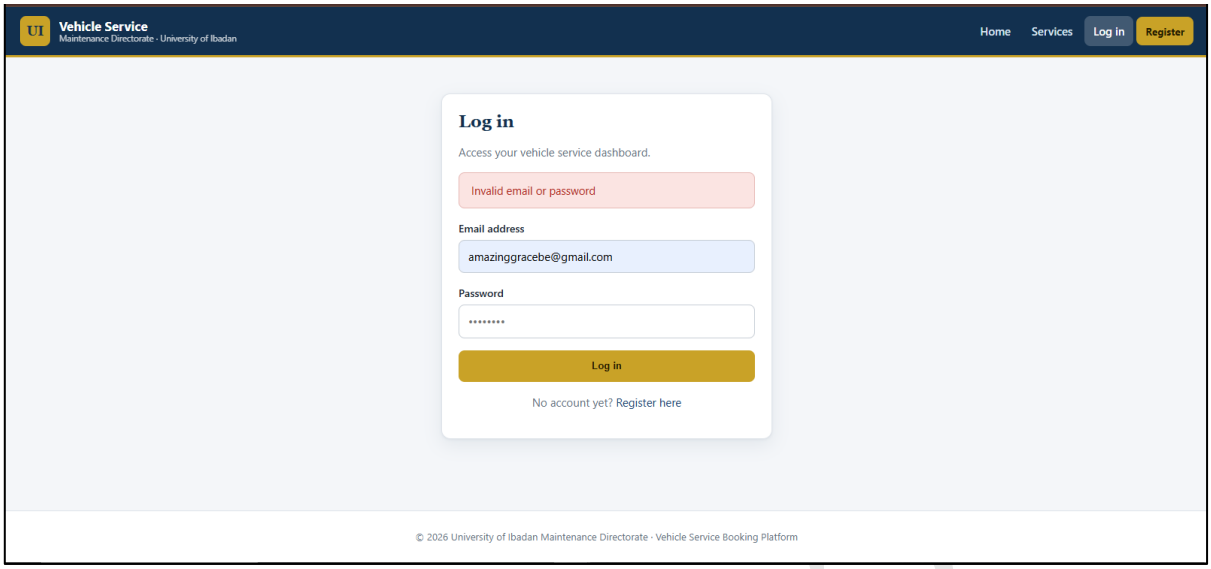


Figure 13.5: Failed login with error message (T05)

13.3 Summary of Results

All 24 test cases were executed and all 24 passed. The successful-path tests (registration, login, vehicle management, booking, payment and admin approval) behaved as expected, and the negative tests (duplicate email, full slot, declined or expired card, unauthorised access and a SQL injection attempt) were correctly rejected with the proper error messages. No failures were recorded, so the system meets its functional requirements.

IFT302

14. Conclusion

The Vehicle Service Booking and Payment Platform replaces phone calls and walk-ins with an online process. Customers register their vehicles, check real-time slot availability, book, pay through a simulated gateway and track progress, while administrators manage services, bookings, payments and records.

The project meets the requirements of the course portfolio: it uses HTML, CSS, JavaScript, PHP and MySQL; the PHP code is organised into object-oriented classes; the Fetch API is used for slot checking, booking and payment; real data is stored in and retrieved from a MySQL database; and payment is fully simulated.

14.1 Limitations

- Payment is simulated and not connected to a real gateway.
- Notifications by SMS or email are not included.
- The slot model uses fixed one-hour slots with a fixed bay capacity.

14.2 Future Work

- Integrate a real payment gateway (for example Paystack in test mode, then live).
- Send email or SMS reminders.
- Add customer cancellation and rescheduling.
- Generate PDF receipts.
- Add reporting dashboards for the workshop.
- Build a mobile app.

IFT302

References

- PHP Manual: https://www.php.net/manual/
- MDN Web Docs, Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MySQL Reference Manual: https://dev.mysql.com/doc/

IFT302